10/29/2026

Rapport de mise en place et d’évaluation du système RAG

SportSee

Aurelie Gabu

Sommaire

Contexte et objectif 2
Environnement de l'application 2
Clé API Mistral 2
Accès PostgreSQL 3
Structure du projet 3
Préparation des données 4
Préparer les documents 4
Nettoyer et découper les documents 4
Construire les chunks et l'index FAISS 4
Création et alimentation de la base PostgreSQL 5
Structure relationnelle 5
Relations entre les tables 5
Principes pour la génération dynamique du SQL 5
Pipeline global 7
Réception et compréhension de la question 8
Récupération des informations 8
Génération de la réponse avec Mistral 8
Validation avec Pydantic 8
Pydantic - validation des données 9
Pydantic AI - encadrement de certaines étapes de traitement 9
Pydantic Logfire - observabilité du pipeline 10
Traitements spécifiques 10
Évaluation comparative avec RAGAS 10
Métriques comparées entre les prototypes 11
Résultats globaux et interprétation 12
Conclusion générale 13
ANNEXES 15

Contexte et objectif

Ce projet consiste à développer et à évaluer un assistant virtuel capable de répondre à des questions sur les *statistiques NBA et les opinions exprimées par les fans dans des discussions Reddit*.

L'assistant combine deux sources de connaissances complémentaires :

- **PostgreSQL / SQL**: récupération de données structurées sur les équipes, les joueurs, les matchs et leurs statistiques
- **RAG (Retrieval-Augmented Generation):** recherche sémantique dans des documents textuels à l'aide d'**`embeddings et de FAISS`**, notamment pour exploiter des discussions Reddit.

L'objectif est double :

- fournir des réponses pertinentes et étayées par les sources disponibles, puis
- mesurer l'impact des améliorations apportées au prototype initial.

Le projet compare de ce fait deux versions du prototype

- **Prototype initial** : version de référence, sans les contrôles et transformations ajoutés au prototype amélioré
- **Prototype amélioré** : version intégrant une validation structurée avec Pydantic, un contrôle sémantique avec Pydantic AI et des traitements spécifiques destinés à fiabiliser les réponses.

L'évaluation compare les deux versions sur un jeu de validation commun à l'aide de métriques **RAGAS**.

Environnement de l'application

Élément	Prérequis
Python	**3.10.8**, version de référence du projet
PostgreSQL	Serveur PostgreSQL accessible et base de données créée
Git	Pour cloner le dépôt
Mistral AI	Compte et clé API valide

Python 3.10.8 est retenu pour reproduire l'environnement utilisé pendant le développement et l'évaluation. La compatibilité avec Python 3.11 ou une version plus récente nécessitera une mise à jour des dépendances.

Clé API Mistral

Une clé API est nécessaire pour :

- générer les réponses ;

- produire les embeddings des documents ;
- réaliser certaines opérations d'évaluation RAGAS ;
- exécuter les contrôles utilisant le modèle Mistral via Pydantic AI.

La clé peut être obtenue depuis la console Mistral : https://console.mistral.ai/

Les appels API peuvent être soumis aux limites de quota et de débit du compte. Les évaluations complètes peuvent donc prendre du temps ou nécessiter une nouvelle exécution en cas d'erreur temporaire.

Accès PostgreSQL

L'utilisateur doit disposer des éléments suivants :

- l'adresse du serveur PostgreSQL ;
- le port de connexion, généralement 5432 ;
- le nom de la base de données ;
- un nom d'utilisateur et un mot de passe ;
- les droits nécessaires pour lire les tables utilisées par le prototype et, selon la procédure d'installation, créer les tables et charger les données.

Structure du projet

```
+---evaluation                          # Set de questions de validation
+---inputs                              # Données brutes
+---ragas_results                       # Résultats d'évaluation
|   +---after_pydantic_ai               # Evaluation après intégration pydantic
|   +---before_pydantic_ai              # Evaluation prototype initial
|   +---prototype_ameliore_pydantic     # Evaluation prototype transformé
+---utils                               # Modules utilitaires
|   +---config_initial.py               # Configuration application initial
|   +---config.py                       # Configuration application amélioré
|   +---data_loader_initial.py          # Préparation des documents
|   +---data_loader_reviewed.py         # Préparation améliorée
|   +---pydantic_validation.py          # Modèles de validation pydantic
|   +---vector_store_initial.py         # Stockage vectoriel initial
|   +---vector_store.py                 # Stockage vectoriel amélioré
+---vector_db                           # Chunks et index FAISS
|__ Comparaison_prototypes.xlsx         # Résumé comparatif prototypes
|__ evaluate_prototypes_pydantic.py     # Evaluation prototype amélioré
|__ evaluation_initiale.py              # Evaluation prototype initial
|__ indexer.py                          # Préparation de l'indexation
|__ load_excel_to_db.py                 # Création du pipeline d'ingestion
|__ MistralChat.py                      # Application streamlit principale
|__ requirements.txt                    # Dépendances
|__ resume_evaluation.py                # Création du fichier de comparaison
|__ sql_tool.py                         # Génération dynamique de requêtes SQL
```

Préparation des données

La préparation des données doit être effectuée avant de lancer l'application ou les évaluations.

Préparer les documents

Les documents sources mis à disposition comprennent des documents PDF contenant des discussions Reddit ou d'autres contenus textuels utiles à l'assistant.

Avant l'indexation, il faut s'assurer que les fichiers sources sont disponibles et que les dépendances nécessaires à leur lecture sont installées.

Nettoyer et découper les documents

Le chargement et le nettoyage des documents sont réalisés par les modules de préparation des données du projet, notamment *data_loader_new.py* pour le prototype amélioré et les fonctions associées.

Les documents sont ensuite découpés en segments appelés *chunks*.

Le chunking a été adapté pour améliorer la qualité des passages récupérés :

- conserver des unités de texte suffisamment cohérentes pour être compréhensibles indépendamment ;
- éviter de transmettre au modèle des documents entiers trop longs ;
- limiter la perte d'informations utiles aux frontières entre deux segments grâce au chevauchement configuré ;
- améliorer la pertinence des passages utilisés pour répondre aux questions qualitatives.

La taille des chunks et leur chevauchement doivent être cohérents avec la configuration réellement utilisée dans le projet. Les modifier implique de reconstruire l'index vectoriel pour que les nouveaux paramètres soient pris en compte.

Construire les chunks et l'index FAISS

Exécuter le script d'indexation : *python indexer.py*

Le processus d'indexation réalise les opérations suivantes :

- lecture des documents sources ;
- extraction et nettoyage du texte ;
- découpage en chunks ;
- génération des embeddings avec Mistral ;
- construction de l'index vectoriel **FAISS** ;
- sauvegarde des chunks et de l'index dans vector_db/.

Les fichiers générés comprennent notamment :

- *vector_db/document_chunks_new.pkl* (prototype amélioré): stockage des chunks utilisés par le prototype ;
- les fichiers de l'index **FAISS**, selon les noms définis dans le module d'indexation.

Il est nécessaire de conserver ensemble les chunks et l'index correspondant. Si les documents ou les paramètres de chunking changent, l'index doit être reconstruit.

Création et alimentation de la base PostgreSQL

La partie **SQL** permet de répondre aux questions quantitatives à partir des données NBA structurées.

La procédure de mise en place comporte deux étapes :

- **Création du schéma** : création des tables, des clés primaires, des clés étrangères et des relations nécessaires. Exécuter le script SQL de création du schéma (*load_excel_to_db.py)*.
- **Chargement des données** : alimentation des tables à partir des données sources.

Le script ne remplace donc pas la création préalable de la base elle-même. Il faut également tenir compte du fait que les tables qu'il gère peuvent être recréées lors de son exécution.

Structure relationnelle

La base est composée de cinq tables principales : **teams, players, matches, stats, top_15_joueurs_points**

La structure générale de la base est la suivante :

- teams <> clé primaire **team_id**
- players <> clé primaire **player_id**, clé étrangère **team_id**
- matches <> clé primaire **match_id**, clé étrangère **player_id**
- stats <> clé primaire **stat_id**, clé étrangère **player_id**
- top_15_joueurs_points <> clé primaire **player_id**

Relations entre les tables

- **teams → players :** Une équipe peut être associée à plusieurs joueurs
  teams.team_id > players.team_id
- **players → matches :** Chaque enregistrement de matches est associé à un joueur
  players.player_id > matches.player_id
- **players → stats :** Chaque enregistrement statistique est associé à un joueur
  players.player_id > stats.player_id
- **players → top_15_joueurs_points :** La table du Top 15 réutilise les identifiants des joueurs
  players.player_id > top_15_joueurs_points.player_id

Principes pour la génération dynamique du SQL

Le **SQL Tool** utilise le schéma de la base ainsi que des exemples **few-shot** pour transformer une question en langage naturel en requête SQL.

Le processus est :

Question utilisateur > Construction du prompt > Ajout du schéma PostgreSQL > Ajout des exemples few-shot > Génération de la requête SQL par le LLM > Validation de la requête > Exécution PostgreSQL > Retour des résultats

Le modèle doit respecter plusieurs règles telles que :

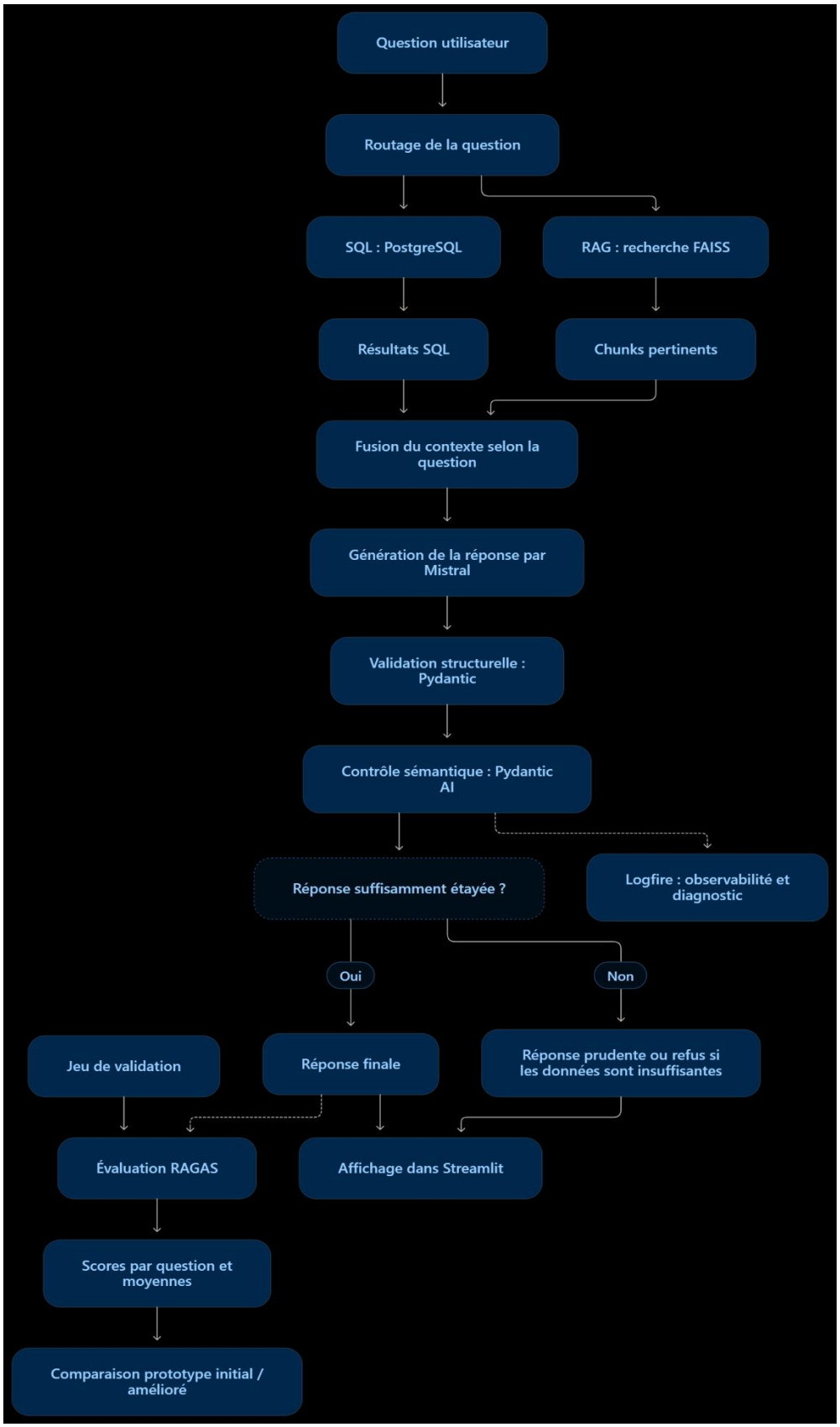
- utiliser uniquement les tables existantes
- utiliser uniquement les colonnes existantes
- utiliser les clés étrangères pour les jointures
- générer uniquement des requêtes de lecture
- ne jamais générer de INSERT, UPDATE, DELETE, DROP, ALTER ou TRUNCATE
- conserver les ex æquo lorsqu'une question porte sur une valeur maximale ou minimale
- utiliser les agrégations (COUNT, AVG, SUM, MIN, MAX) lorsque la question le nécessite.

Après le chargement, vérifier que les tables attendues existent, que les relations entre les joueurs et les équipes fonctionnent et que les statistiques sont renseignées.

La qualité des réponses SQL dépend directement de la qualité, de la couverture et de la structure de ces données.

Pipeline global

Le prototype suit un pipeline hybride combinant récupération de données, génération et contrôle des réponses.



Réception et compréhension de la question

L'utilisateur formule une question en langage naturel.
Le système détermine les sources adaptées à la demande :

- **SQL** pour les statistiques, les classements et les comparaisons quantitatives
- **RAG** pour les opinions, les critiques et les analyses présentes dans les documents
- **SQL + RAG** lorsque la question nécessite de combiner les deux types d'information.

Cette séparation évite de demander au modèle de reconstruire des statistiques à partir de textes lorsqu'une requête SQL peut fournir directement les valeurs.

Récupération des informations

Pour une question **SQL**, *sql_tool.py* exécute la requête appropriée sur **PostgreSQL** et fournit les résultats au générateur de réponse.

Pour une question **RAG**, le système recherche dans **FAISS** les *chunks les plus pertinents* par rapport à la question. Les embeddings permettent de comparer le sens des textes, et pas uniquement leurs mots exacts.

Pour une question **mixte**, les résultats SQL et les passages textuels sont combinés afin de fournir les informations nécessaires à la génération.

Génération de la réponse avec Mistral

Le **modèle Mistral** produit une réponse à partir de la question et du contexte récupéré.

Les résultats SQL font autorité pour les *valeurs statistiques et les classements*.
Les documents RAG servent à étayer les *informations qualitatives*.

Le modèle ne doit pas inventer des statistiques absentes des données, ni présenter une opinion issue de Reddit comme un fait établi.

Le pipeline de génération utilisé lors de l'évaluation a été aligné sur celui du prototype afin de rendre la comparaison plus cohérente : les réponses évaluées doivent correspondre au comportement réel du système, et non à une génération artificiellement différente réservée au benchmark.

Validation avec Pydantic

Pydantic assure la validation structurelle des données échangées dans le pipeline, conformément aux modèles et contraintes définis. Cette validation contribue à limiter les erreurs de format et de cohérence structurelle, mais ne constitue pas une garantie de véracité factuelle des réponses générées.

Un pipeline RAG manipule des données hétérogènes (textes bruts, métadonnées, scores de similarité, embeddings, réponses LLM parfois mal formées), et sans validation, on peut avoir :

- des chunks vides,
- des embeddings corrompus,
- des contextes mal formatés,
- des réponses LLM qui ne respectent pas le format attendu.

Il ne remplace ni PostgreSQL, ni FAISS, ni l'évaluation automatique : il constitue une étape supplémentaire de contrôle avant de retenir la réponse finale.

Pour sécuriser notre pipeline RAG, nous avons intégré trois briques essentielles de l'écosystème Pydantic :

Pydantic - validation des données

Pydantic permet de définir des modèles stricts pour valider automatiquement :

- les chunks,
- les métadonnées,
- les embeddings,
- les réponses LLM.

Cela permet de détecter les données qui ne respectent pas les types, les champs requis et les contraintes définis dans les modèles. Cette validation réduit les risques liés aux données mal formées, sans garantir l'absence de toute erreur dans le pipeline.

Pydantic AI - encadrement de certaines étapes de traitement

Pydantic AI impose un schéma strict aux réponses du LLM. Le modèle doit renvoyer une réponse conforme, sinon une erreur explicite est levée.

Dans le prototype amélioré, Pydantic AI intervient dans les étapes de traitement effectivement intégrées au pipeline, notamment pour encadrer certains contrôles portant sur les réponses générées. Son rôle dépend des mécanismes mis en œuvre et des résultats effectivement exploités par le système. Il peut contribuer à détecter certaines incohérences, mais ne garantit pas à lui seul l'exactitude factuelle des réponses ni l'absence d'hallucinations.

Rôle des modèles existants

Modèle	**Utilisation dans le pipeline**
QuestionModel	Vérifier que la question est renseignée.
ChunkModel	Valider les chunks récupérés par le RAG.
AnswerModel	Vérifier que la question, le contexte et la réponse sont non vides.
ControlledAnswerModel	Structurer le résultat du contrôle : réponse, ancrage dans le contexte et affirmations non justifiées.
RAGResultModel	Construire les données transmises à RAGAS.
DocumentModel, PreparedDocumentModel	Valider les documents avant et après préparation.
Team, Player, Match, Stats, Top15Player	Décrire les données métier, notamment les résultats SQL.

Pydantic Logfire - observabilité du pipeline

Logfire complète le dispositif en facilitant la traçabilité des étapes instrumentées et le diagnostic des erreurs d'exécution. Il s'agit d'un outil d'observabilité, complémentaire à l'évaluation de la qualité des réponses, et non d'un mécanisme garantissant leur exactitude.

Logfire permet d'observer les étapes instrumentées du pipeline, notamment les appels et opérations pour lesquels une instrumentation a été mise en place. Les traces disponibles facilitent le suivi de l'exécution et le diagnostic des erreurs.

Traitements spécifiques

Certaines questions nécessitent un traitement explicite qui ne peut pas être laissé uniquement à la formulation du prompt.

Un exemple important concerne *les classements SQL avec ex æquo*.

La fonction **reconstruire_classement_sql()** a été ajoutée au pipeline pour reconstruire le classement à partir des résultats SQL et conserver les éléments occupant les mêmes rangs, y compris lorsque cela entraîne un nombre de lignes supérieur au nombre demandé.

Cette logique permet d'éviter qu'un classement demandé sur les cinq premières places exclue arbitrairement un élément à égalité à la dernière place.

D'autres contrôles visent à reconnaître les situations où les données disponibles sont insuffisantes. Par exemple, les données agrégées par joueur ou par équipe ne permettent pas nécessairement de répondre à une question portant sur un match précis. De même, une comparaison domicile/extérieur ne peut pas être calculée de façon fiable si les données ne permettent pas d'identifier explicitement ces situations.

Ces traitements ont pour objectif de privilégier une réponse justifiée, ou un refus explicite lorsque cela est nécessaire, plutôt qu'une réponse chiffrée non étayée.

Évaluation comparative avec RAGAS

L'évaluation est réalisée avec **RAGAS** sur un jeu de validation de 31 questions couvrant différents types de demandes :

- statistiques individuelles et collectives ;
- classements et comparaisons ;
- questions nécessitant des informations issues de documents textuels ;
- questions combinant SQL et RAG ;
- questions pour lesquelles les données peuvent être insuffisantes.

Chaque question est associée à une **réponse de référence (ground_truth)** et aux informations nécessaires à l'évaluation.

Pour chaque question, on a notamment :

- l'id de la question,
- la question,

- le type de question (un seul joueur, équipe, comparaison de joueurs, comparaison d'équipes, etc.),
- le niveau de difficulté de la question,
- la présence de bruit,
- la source de données,
- les ground_truth,
- si non résoluble (« Answerable » = true/false)

Métriques comparées entre les prototypes

Métrique	**Ce qu'elle mesure**
answer_relevancy	La pertinence de la réponse par rapport à la question. *« La réponse répond-elle réellement à la question posée ? »*
answer_correctness	La conformité de la réponse à la référence, sur le fond et les faits. *« La réponse produite correspond-elle à la réponse attendue ? »*
context_precision	La pertinence des contextes récupérés, en tenant compte de leur classement. *« Parmi les chunks récupérés, quelle proportion est réellement pertinente pour répondre à la question ?*
context_recall	La couverture des informations attendues dans les contextes récupérés. *« Est-ce que les informations nécessaires pour répondre étaient présentes dans les chunks récupérés ? »*
faithfulness	Le degré auquel les affirmations de la réponse sont étayées par le contexte. *« La réponse produite est-elle soutenue par les informations présentes dans le contexte récupéré ? »*
answer_similarity	La proximité sémantique entre la réponse produite et la référence. *« Dans quelle mesure la réponse et la référence sont-elles sémantiquement proches ? »*
context_entity_recall	La couverture des entités présentes dans la référence par les contextes. *« Quelle proportion des entités présentes dans la référence est retrouvée dans les contextes ? »*

Lors de l'évaluation, pour chaque question on sauvegarde tout ce qui est nécessaire :

- les metadata (les données du validation_set)
- les contexts – les chunks récupérés
- la réponse renvoyée par le modèle
- le ground trugh – la réponse exacte

- les scores

Résultats globaux et interprétation

Les résultats ci-dessous proviennent du tableau de synthèse comparative (*Comparaison_prototypes.xlsx*) fourni dans le projet. Ils sont présentés sur une échelle allant de 0 à 1. Plus le score est élevé, plus le résultat est favorable selon les critères de la métrique considérée. Les scores sont des moyennes sur le jeu de validation

Métrique	Prototype initial	Prototype amélioré
answer_relevancy	0,446	0,723
answer_correctness	0,255	0,729
context_precision	0,140	0,902
context_recall	0,583	0,936
faithfulness	0,535	0,994
answer_similarity	0,839	0,948
context_entity_recall	0,362	0,501

Analyse des résultats RAGAS

- **Pertinence des réponses** : La progression de *answer_relevancy* indique que le prototype amélioré répond plus directement aux questions du jeu de validation.
- **Correction des réponses** : La progression *de answer_correctness* suggère une meilleure conformité aux réponses de référence. Cette évolution est particulièrement importante pour les questions statistiques, les classements et les comparaisons.
- **Qualité des contextes récupérés** : L'évolution de *context_precision* et de *context_recall* indique une amélioration de la pertinence et de la couverture des informations récupérées. Ces métriques doivent néanmoins être interprétées avec prudence lorsque les contextes SQL sont présentés sous forme de résultats tabulaires.
- **Fidélité aux sources** : Le score de *faithfulness* ~= 1,0 dans le tableau comparatif. Selon cette évaluation, les réponses sont étayées par les contextes fournis. Ce résultat ne garantit pas à lui seul que toutes les réponses sont factuellement exactes : le contexte peut être incomplet ou comporter des données erronées.
- **Couverture des entités** : *context_entity_recall* progresse, mais reste inférieur aux autres métriques. Le système peut donc encore manquer certaines entités attendues, notamment lorsque plusieurs joueurs, équipes ou statistiques doivent être comparés.

Analyse des scores à 0.00

Métrique	***Interprétation d'un score de 0.00***

answer_relevancy	La réponse ne répond pas, ou très peu, à la question posée
answer_correctness	La réponse ne correspond pas aux éléments attendus dans la réponse de référence, selon les critères de comparaison de RAGAS
context_precision	Les contextes récupérés sont considérés comme non pertinents pour répondre à la question, selon l'évaluation
context_recall	Aucun des éléments attendus de la réponse de référence n'a été retrouvé dans les contextes, selon l'évaluation
faithfulness	Les affirmations de la réponse ne sont pas étayées par le contexte fourni
answer_similarity	La réponse et la référence présentent une similarité sémantique très faible ou nulle selon la mesure utilisée
context_entity_recall	Aucune des entités attendues dans la référence n'a été retrouvée dans les contextes évalués

Un score de 0,00 ne signifie pas nécessairement que la réponse est entièrement fausse ou inutile. Il signifie que la métrique n'a détecté aucun élément satisfaisant ses critères. Par exemple, une réponse peut être factuellement correcte, mais obtenir un score de pertinence nul si l'évaluateur ne parvient pas à établir son lien avec la question.

De même, ces scores sont des résultats d'évaluation automatique : ils peuvent être influencés par la formulation de la référence, le format du contexte et le comportement du modèle évaluateur. Il faut donc les interpréter conjointement, et non comme des verdicts absolus sur la qualité d'une réponse.

Conclusion générale

Ce projet avait pour objectif de mettre en place un système de génération augmentée par récupération (*Retrieval-Augmented Generation*, RAG) capable de répondre à des questions portant sur les *statistiques NBA* et sur des *documents textuels*, puis d'évaluer l'apport des mécanismes de validation et de contrôle intégrés au prototype amélioré.

L'architecture développée repose sur une approche hybride associant **PostgreSQL** pour l'exploitation des données structurées, **FAISS** pour la recherche documentaire et Mistral pour la génération des réponses. Cette organisation permet d'adapter la récupération du contexte à la nature des questions. Des mécanismes complémentaires de validation avec Pydantic, de traitement avec Pydantic AI et de traçabilité avec Logfire contribuent respectivement à encadrer les données manipulées, les étapes de traitement effectivement intégrées et l'observation de l'exécution.

La comparaison entre le prototype initial et le prototype amélioré, réalisée sur un jeu de 31 questions de référence à l'aide de sept métriques RAGAS, met en évidence une progression sur l'ensemble des indicateurs étudiés. Les améliorations les plus marquées concernent notamment la pertinence du contexte récupéré (*context precision*), la correction des réponses (*answer correctness*) et leur pertinence (*answer relevancy*). La fidélité au contexte (*faithfulness*) atteint également un score de 1,00 dans les résultats obtenus. Ces résultats suggèrent que les

mécanismes introduits contribuent à améliorer les performances du système sur le jeu de test utilisé.

Cette progression doit toutefois être interprétée au regard des limites du protocole. Le benchmark ne comprend que 31 questions et ne couvre pas toutes les formulations ni toutes les situations susceptibles d'être rencontrées en utilisation réelle. Les métriques automatiques RAGAS apportent des indications complémentaires sur différentes dimensions de la qualité, mais leurs scores ne constituent pas une preuve absolue de l'exactitude des réponses. De même, une réponse conforme aux modèles Pydantic peut rester factuellement erronée si le contexte récupéré est incomplet ou si les données disponibles ne permettent pas de répondre à la question.

Les travaux ont également mis en évidence l'importance de traiter explicitement les limites des données et les cas ambigus. Certaines questions nécessitent des informations qui ne sont pas disponibles dans les données agrégées, tandis que d'autres supposent des critères de comparaison qui ne sont pas définis. La gestion des ex æquo dans les classements constitue un autre point d'attention, car une restitution correcte doit préserver les égalités présentes dans les résultats SQL. Ces cas montrent que la qualité d'un système RAG dépend autant de la pertinence de la récupération que de la capacité du système à reconnaître les limites de son contexte et à éviter les conclusions injustifiées.

En conclusion, ce projet a permis de construire une architecture RAG hybride fonctionnelle, de mettre en place un protocole d'évaluation reproductible et de mesurer les effets des améliorations apportées au prototype. Les résultats obtenus sont encourageants sur le benchmark étudié, sans pour autant permettre de généraliser les performances à toutes les questions. Les perspectives d'amélioration portent notamment sur l'enrichissement des données disponibles, l'élargissement du jeu de test, l'analyse des erreurs question par question et le renforcement des contrôles portant sur la justesse factuelle des réponses.

Ce travail souligne ainsi l'intérêt d'associer une architecture de récupération adaptée, des mécanismes de validation et un dispositif d'évaluation mesurable pour améliorer un système de génération augmentée par récupération, tout en conservant une analyse critique de ses résultats et de ses limites.

ANNEXES

Structure de la base de données NBA postgresql

Description du contenu des tables

- **Table teams**

Cette table contient les informations générales relatives aux équipes.

Colonne	Type	Description
team_id	SERIAL	Identifiant unique de l'équipe
code_equipe	CHAR(3)	Code de l'équipe
nom_complet_equipe	TEXT	Nom complet de l'équipe
nombre_joueurs_par_equipe	INT	Nombre de joueurs associés
nombre_points_total_par_equipe	INT	Nombre total de points de l'équipe

Clé primaire

team_id

Contrainte

code_equipe UNIQUE

- **Table players**

Cette table contient les informations principales concernant les joueurs.

Colonne	Type	Description
player_id	SERIAL	Identifiant unique du joueur
team_id	INT	Identifiant de son équipe
nom_du_joueur	TEXT	Nom du joueur
equipe_du_joueur	CHAR(3)	Code de l'équipe
age_du_joueur	INT	Âge du joueur

Clé primaire

player_id

Clé étrangère

team_id → teams.team_id

- **Table matches**

Cette table contient les statistiques liées aux matchs.

Colonne	Type	Description
match_id	SERIAL	Identifiant du match
player_id	INT	Identifiant du joueur
nombre_matchs_joues	INT	Nombre de matchs joués
victoires	INT	Nombre de victoires
defaites	INT	Nombre de défaites
minutes_moyennes	FLOAT	Minutes moyennes jouées
points_moyens	FLOAT	Points moyens par match
tirs_reussis	FLOAT	Tirs réussis
tirs_tentes	FLOAT	Tirs tentés
pourcentage_reussite	FLOAT	Pourcentage de réussite
minutes_apres_15	FLOAT	Minutes jouées après 15 minutes
tirs_3_points_tentes	FLOAT	Nombre de tirs à trois points tentés

Clé primaire

match_id

Clé étrangère

player_id → players.player_id

- **Table stats**

Cette table contient des indicateurs statistiques associés aux joueurs, notamment leur nombre de matchs joués, leurs moyennes de points et de minutes, ainsi que des indicateurs de réussite. Malgré son nom, elle ne permet pas nécessairement d'accéder aux statistiques de chaque match individuellement. Les analyses par match nécessitent des données enregistrées à cette granularité.

Colonne	Type	Description
stat_id	SERIAL	Identifiant de la statistique
player_id	INT	Identifiant du joueur
pourcentage_3_points	FLOAT	Pourcentage à trois points
lancers_francs_reussis	FLOAT	Lancers francs réussis

Colonne	Type	Description
lancers_francs_tentes	FLOAT	Lancers francs tentés
pourcentage_lancers_francs	FLOAT	Pourcentage de lancers francs
rebonds_offensifs	FLOAT	Rebonds offensifs
rebonds_defensifs	FLOAT	Rebonds défensifs
rebonds_totaux	FLOAT	Rebonds totaux
passes_decisives	FLOAT	Passes décisives
balles_perdues	FLOAT	Balles perdues
interceptions	FLOAT	Interceptions
contres	FLOAT	Contres
fautes_personnelles	FLOAT	Fautes personnelles
fantasy_points	FLOAT	Fantasy points
double_doubles	INT	Nombre de double-doubles
triple_doubles	INT	Nombre de triple-doubles
plus_minus	FLOAT	Plus/minus
offensive_rating	FLOAT	Offensive rating
defensive_rating	FLOAT	Defensive rating
net_rating	FLOAT	Net rating
pourcentage_assists	FLOAT	Pourcentage de passes décisives
ratio_assists_pertes	FLOAT	Ratio passes décisives / pertes
ratio_assists_100_possessions	FLOAT	Passes décisives pour 100 possessions
pourcentage_rebonds_offensifs	FLOAT	Pourcentage de rebonds offensifs
pourcentage_rebonds_defensifs	FLOAT	Pourcentage de rebonds défensifs
pourcentage_rebonds_totaux	FLOAT	Pourcentage de rebonds totaux
turnover_ratio	FLOAT	Turnover ratio

Colonne	Type	Description
efg_percent	FLOAT	Effective field goal percentage
true_shooting_percent	FLOAT	True shooting percentage
usage_rate	FLOAT	Usage rate
rythme_de_jeu	FLOAT	Rythme de jeu
player_impact_estimate	FLOAT	Estimation de l'impact du joueur
possessions_totales	FLOAT	Nombre total de possessions

Clé primaire

stat_id

Clé étrangère

player_id → players.player_id

- **Table top_15_joueurs_points**

Cette table contient les joueurs appartenant au Top 15 des joueurs sur différentes statistiques.

Colonne	Type	Description
player_id	INT	Identifiant du joueur
nom_du_joueur	TEXT	Nom du joueur
nombre_points_total	FLOAT	Nombre total de points
tirs_reussis	FLOAT	Tirs réussis
pourcentage_tirs_reussis	FLOAT	Pourcentage de tirs réussis
pourcentage_tirs_3_points	FLOAT	Pourcentage à trois points
pourcentage_lancers_francs	FLOAT	Pourcentage de lancers francs
rebonds_offensifs	FLOAT	Rebonds offensifs
impact_estime	FLOAT	Impact estimé du joueur

Clé primaire

player_id

Clé étrangère

player_id → players.player_id

Cette table est utilisée lorsqu'une question porte spécifiquement sur le classement des meilleurs marqueurs.

Exemples de requêtes types

✓ **Rechercher le joueur ayant marqué le plus de points**

```
SELECT
  p.nom_du_joueur,
  t.nombre_points_total
FROM top_15_joueurs_points t
JOIN players p
  ON t.player_id = p.player_id
WHERE t.nombre_points_total = (
  SELECT MAX(nombre_points_total)
  FROM top_15_joueurs_points)
ORDER BY p.nom_du_joueur;
```

L'utilisation de la fonction MAX() permet de conserver les éventuels ex æquo.

✓ **Rechercher le(s) joueur(s) le(s) plus jeune(s) et le(s) plus âgé(s)**

```
SELECT
  nom_du_joueur,
  age_du_joueur
FROM players
WHERE age_du_joueur = (
  SELECT MIN(age_du_joueur)
  FROM players)
OR age_du_joueur = (
  SELECT MAX(age_du_joueur)
  FROM players)
ORDER BY age_du_joueur;
```

Cette requête retourne tous les joueurs correspondant aux deux âges extrêmes.

✓ **Moyenne de points par équipe**

```
SELECT
  t.nom_complet_equipe,
  AVG(m.points_moyens) AS moyenne_points
FROM teams t
JOIN players p
  ON t.team_id = p.team_id
JOIN matches m
  ON p.player_id = m.player_id
GROUP BY
  t.team_id,
  t.nom_complet_equipe
ORDER BY moyenne_points DESC;
```

✓ **Plusieurs indicateurs par équipe**

```
SELECT
  t.nom_complet_equipe,
  COUNT(DISTINCT p.player_id) AS nombre_joueurs,
  AVG(m.points_moyens) AS moyenne_points,
  AVG(m.pourcentage_reussite) AS moyenne_pourcentage_reussite,
  AVG(s.rebonds_totaux) AS moyenne_rebonds
FROM teams t
JOIN players p
  ON t.team_id = p.team_id
JOIN matches m
  ON p.player_id = m.player_id
JOIN stats s
  ON p.player_id = s.player_id
GROUP BY t.team_id, t.nom_complet_equipe
ORDER BY moyenne_points DESC;
```

Cette requête constitue un exemple d'**agrégation multicritère**.

✓ **Comparaison domicile / extérieur**

La base actuelle ne contient pas de colonne permettant d'identifier directement le contexte **domicile / extérieur** dans les tables présentées.

Si une variable de ce type est ajoutée aux données avec les valeurs Domicile/Extérieur, une comparaison pourrait être réalisée avec :

```
SELECT
  domicile_exterieur,
  AVG(points_moyens) AS moyenne_points,
  AVG(pourcentage_reussite) AS moyenne_reussite
FROM matches
GROUP BY domicile_exterieur;
```

Pour comparer les performances d'un joueur :

```
SELECT
  p.nom_du_joueur,
  m.domicile_exterieur,
  AVG(m.points_moyens) AS moyenne_points
FROM players p
JOIN matches m
  ON p.player_id = m.player_id
WHERE p.nom_du_joueur = 'Nom du joueur'
GROUP BY
  p.nom_du_joueur,
  m.domicile_exterieur;
```

Remarque : ces requêtes sont des modèles génériques. Elles ne peuvent être exécutées que si la colonne domicile_exterieur existe effectivement dans la base.

✓ **Joueurs ayant plus de 20 points et plus de 5 rebonds**

```
SELECT
  p.nom_du_joueur,
  m.points_moyens,
  s.rebonds_totaux
FROM players p
JOIN matches m ON p.player_id = m.player_id
JOIN stats s  ON p.player_id = s.player_id
WHERE m.points_moyens > 20 AND s.rebonds_totaux > 5
ORDER BY m.points_moyens DESC,  s.rebonds_totaux DESC;
```

Principes pour la génération dynamique du SQL

Le **SQL Tool** utilise le schéma de la base ainsi que des exemples **few-shot** pour transformer une question en langage naturel en requête SQL.

Le processus est :

Question utilisateur > Construction du prompt > Ajout du schéma PostgreSQL > Ajout des exemples few-shot > Génération de la requête SQL par le LLM > Validation de la requête > Exécution PostgreSQL > Retour des résultats

Le modèle doit respecter plusieurs règles telles que :

- utiliser uniquement les tables existantes
- utiliser uniquement les colonnes existantes
- utiliser les clés étrangères pour les jointures
- générer uniquement des requêtes de lecture
- ne jamais générer de INSERT, UPDATE, DELETE, DROP, ALTER ou TRUNCATE
- conserver les ex æquo lorsqu'une question porte sur une valeur maximale ou minimale
- utiliser les agrégations (COUNT, AVG, SUM, MIN, MAX) lorsque la question le nécessite.

Exemples de questions destinées au SQL Tool

Les questions suivantes peuvent être utilisées pour tester la génération dynamique des requêtes :

Catégorie	**Question**
Classement	Quel joueur a marqué le plus de points ?
Classement	Quels sont les 5 meilleurs marqueurs ?
Minimum / maximum	Quel est le joueur le plus jeune ?
Minimum / maximum	Quel est le joueur le plus jeune et le plus âgé ?
Ex æquo	Quels joueurs ont le meilleur pourcentage à trois points ?

Catégorie	Question
Filtrage	Quels joueurs ont marqué plus de 20 points par match ?
Agrégation	Combien de joueurs appartiennent à chaque équipe ?
Agrégation	Quelle est la moyenne de points par équipe ?
Multicritère	Quels joueurs ont plus de 20 points et plus de 5 rebonds ?
Multitable	Quelle équipe possède les joueurs ayant la meilleure moyenne de points ?
Statistiques	Quel joueur possède le meilleur offensive rating ?
Statistiques	Quels joueurs ont réalisé au moins un triple-double ?
Comparaison	Quelle équipe possède le meilleur pourcentage de réussite moyen ?
Domicile / extérieur	Les performances sont-elles différentes à domicile et à l'extérieur ?*

* Cette dernière question nécessite une variable permettant d'identifier le contexte domicile/extérieur dans les données.

Sécurité des requêtes

Le SQL Tool ne doit pas permettre au modèle de modifier la base de données.

Les commandes suivantes sont interdites :

- INSERT
- UPDATE
- DELETE
- DROP
- ALTER
- CREATE
- TRUNCATE
- GRANT
- REVOKE

Le SQL Tool accepte uniquement les requêtes de lecture commençant par : SELECT ou WITH

Une validation est effectuée avant l'exécution de la requête.

Cette validation constitue une mesure de sécurité complémentaire ; elle ne remplace pas une gestion stricte des permissions PostgreSQL.